

Evaluating AI-Assisted Software Development Practices in Student Projects: A Comparative Workshop-Based Study

Sahan Fernando^a, Second Author^a, Third Author^a

^aDepartment of Industrial Management, Faculty of Science, University of Kelaniya

**Bhstfe261@kln.ac.lk*

Introduction: AI coding tools are used in student projects, challenging supervision, assessment, and quality assurance. Generated applications may appear demo-ready while hiding weaknesses in requirements, database use, login and role control, testing, security, and maintainability.

Objectives: This study aimed to: (1) compare software project outputs produced under three AI-use conditions; and (2) identify assessment evidence academic staff need when evaluating AI-assisted student projects.

Methods: The workshop used 10 predefined client-style cases constrained to React, Node.js/Express, and local MySQL. Each case produced three implementations under distinct AI-use conditions: highly AI-dependent prompting with short casual prompts, software engineering-aware prompting with concise technical prompts, and engineering-led prompting with software and prompt engineering guidance. All followed checkpoints covering interpretation, architecture, data modelling, UI, core workflow, login and role control, secondary features, testing, security, maintainability, change requests, and final review. Outputs were evaluated through shared mid-review, final-review, and comparative code-evidence matrices using a 0-5 scale across 24 feature areas and seven weighted categories.

Results: Comparative reports indicated that engineering-led prompting produced the strongest evidence of requirements alignment, backend authorization, documentation, testing discipline, and change handling. AI-dependent outputs could achieve broad visible functionality and, in some areas, strong workflow coverage; however, recurring weaknesses included weak login design, plaintext or exposed credentials, large code files, and incomplete server-side workflow rules. Software engineering-aware outputs generally improved code organization, testing approach, and evidence for preventing role misuse, but did not automatically preserve case-specific workflow rules. Even the strongest outputs retained security or edge-case gaps requiring inspection.

Discussion: Findings show that interface demonstrations are insufficient for evaluating AI-assisted student projects. Server routes, database design, configuration, test evidence, prompt history, and viva questioning reveal critical hidden weaknesses.

Conclusions and recommendations: The workshop model helps academic staff distinguish generated functionality from engineered software and provides a reusable framework for AI-aware SDP assessment.

Keywords: AI-assisted coding; software engineering education; prompt engineering; student project assessment; digital education.
